PyGame Flappy Bird Tutorial - Part 7

Contents

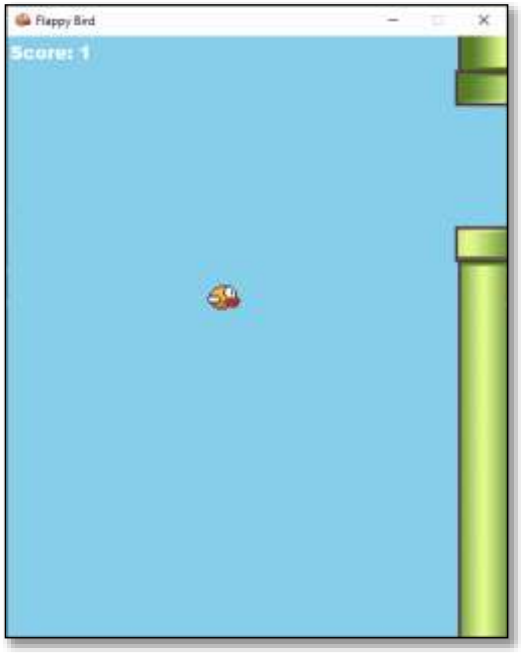
PyGame Flappy Bird Tutorial - Part 7 1
Preview of the Game 1
Sounds 1
Full Game 2
What's Next? 7
Assignment Submission 7

Time required: 30 minutes

Preview of the Game

Here's a sneak peak of the game that we are going to work on.

Flappy Bird Demo Video



Sounds

You can use the sounds in the asset file or create your own.

- https://www.beepbox.co (Create 8-bit songs.)
- https://sfxr.me/ (Create sound effects.)
- https://elevenlabs.io/sound-effects
- https://www.leshylabs.com/apps/sfMaker

Full Game

Time to finish up the game with scoring and sounds.

1. Save **flappy_bird_6.py** as **flappy_bird_7.py**
2. Modify the existing code.
3. Add sleep to pause the game for a moment.

```
"""
    Name: flappy_bird_7.py
    Author:
    Date:
    Purpose: Flappy Bird Clone in OOP
"""
# pip install pygame-ce
# Import pygame library
import pygame
# pip install pygame-menu
import pygame_menu as pm
# Import exit for a clean program shutdown
from sys import exit
from random import randint
from time import sleep
import config
```

4. Initialize the pygame mixer to optimize the sounds.

```python
class FlappyBird:
    def __init__(self):
        # pre initalize mixer with larger buffer size for better performance
        pygame.mixer.pre_init(
            44100,  # frequency (Hz)
            16,     # bit depth
            2,      # number of channels, 1 mono, 2 stereo
            4096    # buffer size, larger to optimize music playback.
        )

        # Initialize pygame engine
        pygame.init()
```

5. Load the background music, set the volume and start the music playing. Setup a few variables.

```python
        self.init_bird()
        self.init_pipes()

        # Load flappy bird program png icon
        self.bird_ico = pygame.image.load(
            "./assets/flappy_bird_ico.png").convert_alpha()

        pygame.display.set_icon(self.bird_ico)

        # Load background music file into memory
        pygame.mixer.music.load('./assets/flying-minimal.mp3')

        # Set volume to 30%, range from 0.0 (mute) to 1.0 (full volume)
        pygame.mixer.music.set_volume(0.3)

        # Play in a loop until stopped
        pygame.mixer.music.play(-1)

        self.score = 0
        self.game_over = False
        self.pass_pipe = False
        self.score_font = pygame.font.SysFont("arialblack", 18)
```

6. Add score counted variable to the end of the init pipes method.

```
        # Set lower pipe vertical location
        self.pipe_lower_rect.top = self.pipe_upper_rect.bottom \
            + self.pipe_gap_size

        # Set score counted to false
        self.score_counted = False
```

7. Add the same variable to the end of the reset pipes method.

```
# Set initial X off screen to right
self.pipe_upper_rect.left = config.WIDTH
self.pipe_lower_rect.left = config.WIDTH

# New set of pipes, reset score counter
self.score_counted = False
```

8. Add the display score method.

```
# ------------------ DISPLAY SCORE -------------------------------------#
    def display_score(self):
        """Display the score on the screen"""
        # Create text image for score display
        text = self.score_font.render(f"Score: {self.score}", True, "white")
        self.surface.blit(
            text,   # Image to display
            [3, 3]  # x , y to display the image
        )
```

9. Make some modifications to the display game over menu.

```python
# ------------------------- DISPLAY GAME OVER ----------------------------#
    def display_game_over(self):
        """Display game over menu using the Pygame Menu library"""
        # Stop background sound
        pygame.mixer.music.stop()

        # Play crash sound
        crash = pygame.mixer.Sound('./assets/crash_short.mp3')
        crash.play()
        crash.set_volume(0.3)

        # Wait 2 second while crash plays
        sleep(2)

        # Define a menu object for the game over screen
        game_over = pm.Menu(
            title="Game over",      # Set title menu to "Game over"
            width=config.WIDTH,     # Set to width of game surface
            height=config.HEIGHT,   # Set to height of game surface
            # Set the theme of the menu to an orange color scheme
            theme=pm.themes.THEME_ORANGE
        )

        # Display final score
        game_over.add.label(f"Score: {self.score}")

        # Add label to provide space between buttons
        game_over.add.label("")

        # Add a button to the game over menu for exiting the game
        game_over.add.button(
            title="Play Again?",    # Button text
            action=main             # Call main() to start over
        )

        # Add label to provide space between buttons
        game_over.add.label("")

        # Add a button to the game over menu for exiting the game
        game_over.add.button(
            title="Exit",           # Button text
            action=pm.events.EXIT   # Exit the game when clicked
        )

        # Run the main loop of the game over menu on the specified surface
        game_over.mainloop(self.surface)
```

10. Some game_loop modifications.

```python
# --------------------------- GAME LOOP ----------------------------------#
    def game_loop(self):
        """Infinite game loop"""
        while not self.game_over:
            self.check_events()
            self.detect_collision()
            # Simulate gravity by moving the bird down
            # unless the UP key is pressed
            # Reset gravity to 3 each time through the loop
            gravity = 3

            # Get list of keys being pressed
            key_input = pygame.key.get_pressed()

            # If up cursor pressed, move up 5 pixels
            if key_input[pygame.K_UP]:
                gravity -= 5

            # ------------------- INCREASE DIFFICULTY ---------------------#
            # Adding difficulty relative to score
            # Increase the speed and decrease the gap of blocks
            if 5 <= self.score < 10:
                self.pipe_move = 5
                self.pipe_gap_size = self.bird_rect.height * 4

            elif 10 <= self.score < 20:
                self.pipe_move = 7
                self.pipe_gap_size = self.bird_rect.height * 3.5

            # ---------------------- SCORING ------------------------------#
            # If the bird makes it past the pipes, increase score
            if self.bird_rect.left > self.pipe_upper_rect.right \
                    and not self.score_counted:

                # Increase score
                self.score += 1

                # Track whether the current set of pipes have had a score
                self.score_counted = True
```

Example run:

A complete game!

What's Next?

There is much more that can be done with this game. Here are some ideas for you to practice and implement on your own.

- Keep track of the score between games.
- Change how often and how many pillars come along.
- Change the pillar image.
- Add some additional audio to the game, such as movement sounds (audio that plays when you move the character)
- Add the concept of multiple Lives or a Health bar.
- Change the colors.
- Change the game to make it your own.

Assignment Submission

1. Attach a screenshot showing the operation of the program.
2. Zip up the program files folder and submit in Blackboard.